Project Sentinel — Architecture & Code Map

This document maps the evaluation pipeline from a GitHub link through extraction, analysis, specialist agents, deterministic scoring, narrative generation, and final report generation. Each step lists the implementing file and a short code snippet.

API: accept GitHub link

File: backend/main.py

```
safe_github_url = validate_github_url(github_url)
project = Project(..., github_url=safe_github_url, ...)
db.add(project); db.commit()
return JSONResponse({"project_id": project_id, ...})
```

Trigger evaluation (background)

File: backend/main.py

```
background_tasks.add_task(_run_evaluation_background, project_id)
# background worker: results = run_evaluation(project_id, ctx, ctx_text)
```

Fetch + extract repository data

File: backend/tools/github_tool.py

```
def extract_github_data(github_url: str) -> dict[str, Any]:
    repo_name = _repo_name_from_url(github_url)
    gh = _github_client()
    repo = gh.get_repo(repo_name)
    result['readme'] = _safe_decode(repo.get_readme())[:6000]
    file_paths = _tree_file_paths(repo)
```

Build project context & code intelligence

File: backend/tools/parser.py

```
futures['github'] = executor.submit(extract_github_data, github_url)
ctx['code_reader'] = read_codebase(ctx)
ctx['architecture'] = summarize_architecture(ctx, ctx['code_reader'])
```

Static security scanning

File: backend/tools/parser.py -> tools/security_tool.py

```
security_future = executor.submit(run_security_analysis, python_files, dep_files, source_files)
ctx['security'] = security_future.result()
```

Build deterministic brief

File: backend/eval_context/brief_builder.py

```
brief: EvaluationBrief = build_brief(ctx)
brief_text = truncate_brief(brief.to_text())
```

Slice context per-agent

File: backend/eval_context/context_slicer.py

```
def slice_context_for_agent(ctx, agent):
    if agent == 'technical': brief_parts.append(_gh_excerpt(ctx, readme_limit=1000))
```

```
    return truncate_text(text, MAX_AGENT_DIGEST_CHARS, label=f'digest:{agent}')
```

Specialist task factories

File: backend/tasks/evaluation_tasks.py

```
def create_technical_task(agent, brief, digest):
    return Task(description=f'Brief:
{brief}
Evidence:
{digest}
Return JSON: {"technical_score": <int 0-100>, ... }')
```

Specialist agent factory

File: backend/agents/specialist_agents.py

```
def _agent(...):
    return Agent(..., llm=get_llm('specialist'), max_iter=1, max_execution_time=600)
```

Run specialists and parse JSON

File: backend/crew/crew.py

```
raw = invoke_with_retry(lambda: _execute(), fallback_fn=_execute_fallback)
report, parse_source = parse_agent_json(raw, model_cls, FALLBACKS[name])
```

Deterministic scoring in Python

File: backend/scoring/score_engine.py

```
overall = round(sum(agent_map[k] * WEIGHTS.get(k,0) for k in agent_map))
if overall >= 85: verdict='APPROVE' ... return {'overall_score': overall, 'verdict': verdict, 'risk_level': ris
```

Narrative agent (LLM) - narrative-only

File: backend/agents/narrative_agent.py & backend/tasks/evaluation_tasks.py

```
create_narrative_task(agent, numeric_summary, key_findings) -> Task(description='Numeric summary:
{...}
Key findings...')
```

Invoke narrative & fallback

File: backend/crew/crew.py

```
narrative_raw = _run_agent_llm(agent=narrative_agent, task=narrative_task, ...)
if not narrative_raw.strip(): narrative_raw = json.dumps({'executive_summary': 'Evaluation completed successful
```

Report generation (PDF)

File: backend/reports/report_generator.py

```
path = generate_report(project.name, project.id, results)
```

Notes

PDF generated programmatically from repository files. If ReportLab is not installed, install via pip install reportlab.